# Privacy-Preserving Federated Learning for IoT-Based Healthcare: A Layered-Defense Benchmark on ECG Arrhythmia Classification

Mapeke

2026

**Abstract**

Federated learning (FL) is the textbook answer to privacy-preserving training on healthcare IoT data, and the layered defenses around it — DP-SGD, secure aggregation, and update compression — are textbook ingredients of a "privacy-preserving FL" pipeline. The textbook does not say which layer pays for which threat, or what each one costs in detection utility on a realistic clinical workload. This thesis builds a fully working layered-defense framework in roughly 2,500 lines of Python (Flower + PyTorch + Opacus), and benchmarks it on the MIT-BIH Arrhythmia dataset with four simulated wearable IoT clients training a 1-D CNN over the AAMI 5-class label scheme. Three findings drive the thesis: (1) federation alone is essentially free — vanilla FedAvg matches a centralized baseline within 0.02 pp test accuracy on this workload; (2) pairwise-mask secure aggregation is also free — it shifts test accuracy by 0.20 pp, well below seed variance, and the cryptographic correctness property (mask cancellation under summation) is encoded as a unit test that runs in CI; (3) DP-SGD at $\varepsilon = 5$ collapses the model to the majority-class predictor on this heavily imbalanced workload, with final accuracy equal to the prevalence of class $N$ to two decimals — a privacy/utility cliff that is invisible if one only reports overall accuracy. Adding compression on top of DP does not degrade the model further, because there is nothing left to degrade. A follow-up run on MIT-BIH records 200–234 (where the imbalance is moderate, 82 % N rather than 97 %) shows that this collapse is dataset-conditional: DP-SGD alone retains a +6.37 pp signal over the majority baseline on that slice, but the full pipeline (DP + SecAgg + compression at $\varepsilon = 8$) still collapses to majority. The cliff is real; its location is a property of the workload's class balance. Every number in the thesis is reproducible from a committed YAML configuration via `python -m experiments.run_*`.

# Contents


**1 Introduction** **3**
1.1 Motivation . . . 3
1.2 Contributions . . . 3
1.3 Structure . . . 4
1.4 Scope and non-goals . . . 4

**2 Background** **5**
2.1 Federated learning and FedAvg . . . 5
2.2 The privacy stack . . . 5
2.3 Dataset and use case . . . 6

**3 Threat model** **7**
3.1 Assets being protected . . . 7
3.2 Adversaries and capabilities . . . 7
3.2.1 A1 — Honest-but-curious aggregation server . . . 7
3.2.2 A2 — Curious-client coalition (size $t < N$) . . . 8
3.2.3 A3 — Network eavesdropper . . . 8
3.2.4 A4 — External adversary holding the released global model . . . 8
3.2.5 A5 — Malicious (Byzantine) client . . . 9
3.3 Attack-surface coverage matrix . . . 9
3.4 Trust assumptions . . . 10
3.5 What is explicitly out of scope . . . 10

**4 Method** **11**
4.1 Unified client interface (`src/client/flower_client.py`) . . . 11
4.2 Data layer (`src/data/`) . . . 11
4.3 Models (`src/models/`) . . . 11
4.4 Differential privacy (`src/privacy/differential_privacy.py`) . . . 12
4.5 Secure aggregation (`src/privacy/secure_agg.py`) . . . 12
4.6 Compression (`src/privacy/compression.py`) . . . 12
4.7 Server strategy (`src/server/strategies.py`) . . . 12
4.8 Experiment infrastructure . . . 12
4.9 Tests . . . 13

**5 Results I — Centralized and federated baselines** **14**
5.1 Detection grid . . . 14
5.2 Centralized baseline . . . 14
5.3 Vanilla FedAvg . . . 14
5.4 Reproducibility check vs. published FL ECG results . . . 15

**6 Results II — Secure aggregation** **16**
6.1 Mask cancellation as a code-level invariant . . . 16
6.2 End-to-end SecAgg run . . . 16
6.3 What SecAgg does not protect against . . . 16

**7 Results III — Differential privacy and the full pipeline** **18**
7.1 DP-SGD run . . . 18
7.2 Class-prior collapse . . . 18
7.3 The full pipeline run . . . 19
7.4 Negotiating the cliff . . . 19
7.5 Follow-up: records 200–234 . . . 19

**8 Discussion** **22**
8.1 What the three findings add up to . . . 22
8.2 Likely reviewer objections . . . 22
8.3 Practical implications for deployment . . . 23
8.4 What we deliberately did not test . . . 23
8.5 Future work . . . 23

**9 Conclusion** **25**

# Chapter 1

# Introduction

## 1.1 Motivation

Connected health devices — smartwatches, Holter monitors, bedside telemetry, the implantable devices that already exist in patients — produce streams of physiological data that are simultaneously *clinically valuable* and *highly sensitive*. Existing deployments centralize that data, which (i) creates a single point of failure for breaches, (ii) exposes patients to re-identification even after pseudonymization, and (iii) conflicts with the data-minimization principle of HIPAA and GDPR. Federated learning was proposed [1] precisely to break this pattern: each device trains locally on its own raw data, the central server only ever sees model updates.

The naive version of this story is incomplete in two ways that the deep-learning literature is comfortable papering over:

1. **Model updates leak.** Per-client gradients reconstruct training records under known attacks; a coalition of clients or an honest-but-curious server can recover patient data from the updates alone. Differential privacy [2] and secure aggregation [4] are the two canonical defenses, but each addresses a different threat and each introduces its own utility tax.

2. **Healthcare data is heavily imbalanced.** ECG arrhythmia classification on MIT-BIH is roughly 97 % normal beats; rare classes can be $< 0.1\,\%$. Privacy mechanisms designed and tuned on balanced datasets (CIFAR, MNIST) behave badly here in ways the headline-accuracy literature rarely surfaces.

This thesis takes both of these seriously. We build a working layered-defense framework, run it end-to-end on real PhysioNet data, and report what each privacy layer actually costs — including the cases where the cost is the detector ceasing to detect anything.

## 1.2 Contributions

**C1 — A layered-defense FL framework with separable privacy layers.** The Flower client interface used here exposes three independent switches (differential privacy, secure aggregation, compression) which can be enabled in isolation or jointly via a single YAML config. The marginal cost of each layer on test accuracy, spent $\varepsilon$, and convergence is measured by holding the others fixed (Chapters 5, 6, 7). The implementation is roughly 2,500 lines of Python and runs end-to-end on a single workstation in under 30 minutes.

**C2 — Cryptographic correctness as a code-level invariant.** The pairwise-mask secure aggregation protocol's correctness argument (every mask appears with + and − exactly once, so summing client updates cancels every mask) is encoded as a unit test (`tests/test_secagg.py::test_masks_can` that runs in CI. The argument from `docs/privacy-analysis.md` §2 [4] is therefore mechanically checked rather than prose-only.

**C3 — A class-prior collapse diagnostic for DP on imbalanced healthcare data.** On the MIT-BIH workload used here, the DP-SGD run at $\varepsilon \approx 5$ converges to a final test accuracy of **0.9711**, which equals the prevalence of class $N$ (Normal) to four decimals. The full-pipeline run (DP + SecAgg + compression) at $\varepsilon \approx 8$ converges to the *same* 0.9711. We argue (Chapter 7, §7.2) that this is a class-prior collapse, not a learning curve, and that it is the single most consequential and least discussed failure mode of DP-SGD on realistic clinical data. A follow-up on records 200–234 (§7.5) confirms the diagnostic generalises but its severity is class-balance-conditional: on moderate imbalance DP-SGD retains a measurable rare-class signal, while the full pipeline still collapses.

## 1.3 Structure

Chapter 2 reviews FL, the three privacy primitives we layer, and the MIT-BIH dataset. Chapter 3 formalises the threat model: the assets being protected, five adversaries A1–A5 with their capabilities and goals, the attack-surface coverage matrix, and the trust assumptions the framework inherits. Chapter 4 describes the implementation: `FlowerHealthcareClient`, `PrivacyAwareFedAvg`, `DPConfig+attach_dp`, the pairwise-mask SecAgg primitives, and the top-$k$/int8 compression pipeline. Chapter 5 reports the centralized baseline and vanilla FedAvg. Chapter 6 adds secure aggregation. Chapter 7 adds differential privacy and the full pipeline. Chapter 8 discusses anticipated reviewer objections and limitations. Chapter 9 concludes.

## 1.4 Scope and non-goals

- **We do not propose a new privacy primitive.** The argument is empirical: existing primitives composed under a protocol that makes their individual costs visible.
- **We do not deploy on real IoT hardware.** Every number is from a Flower simulation on a single GPU. The IoT angle is in the data partitioning (per-patient non-IID) and in the device-profile metadata (`wearable`, `bedside`, `gateway`); the experiments do not exercise actual network or compute constraints.
- **We do not evaluate against poisoning / Byzantine clients.** The threat model is honest-but-curious server plus an external adversary holding the released global model (Appendix `docs/threat-model.md`). Robust aggregation (Krum, median, trimmed mean) is listed as future work (§8.5).
- **Our primary experiments use MIT-BIH records 100–115**, which `wfdb.dl_database` pulls down by default and which exhibit the heavy-imbalance regime (97.1 % N) that drives the central finding. A follow-up section (Chapter 7, §7.5) re-runs all five experiments on records 200–234 (more arrhythmia variety, 82.1 % N) to test whether the DP class-prior collapse is dataset-conditional.

# Chapter 2

# Background

## 2.1 Federated learning and FedAvg

The federated learning protocol of McMahan et al. [1] distributes training across $N$ clients each holding a local dataset $D_i$, with the server orchestrating $T$ rounds of:

$$w_{t+1} \;=\; \sum_{i=1}^{N} \tfrac{n_i}{n}\,\mathrm{LocalSGD}(w_t, D_i),$$

where $n_i = |D_i|$ and $n = \sum_i n_i$. Raw $D_i$ never leaves client $i$; only $\mathrm{LocalSGD}(w_t, D_i)$ does. FedAvg is the baseline we layer defenses on top of.

The healthcare specialization is straightforward: each $D_i$ is a per-patient or per-device partition of physiological data. We use four simulated wearable clients holding disjoint Dirichlet-non-IID partitions (§4.2) of MIT-BIH ECG beats.

## 2.2 The privacy stack

Three layers, each addressing a different threat.

**Differential privacy (DP-SGD).** Replaces every local SGD step with: clip per-sample gradients to $\ell_2$-norm $C$, add Gaussian noise of scale $\sigma C$, then step. Composing across $T$ rounds and converting to $(\varepsilon, \delta)$ via the RDP accountant [3] gives a tight bound on what an adversary holding only the released global model can learn about any single training record. Implemented via Opacus [5].

**Pairwise-mask secure aggregation.** For every unordered client pair $\{i, j\}$, both clients derive a shared mask vector $m_{ij}$. Client $i$ adds $+m_{ij}$ to its update; client $j$ adds $-m_{ij}$. The server's sum cancels every mask exactly, so the server only ever sees the aggregate $\sum_i \Delta w_i$ and never any individual $\Delta w_i$ [4]. Defends against an honest-but-curious server that would otherwise inspect per-client updates.

**Top-$k$ + int8 compression.** Each client keeps the largest-magnitude $k$ fraction of its update entries and quantizes them into 8-bit integers. Reduces bandwidth roughly $40\times$ for $k = 0.1$. Post-processing-immune to DP and compatible with SecAgg as long as both endpoints agree on the sparsification mask.

**Composition.** DP and SecAgg compose without interaction: the DP guarantee is on $w_T$ regardless of channel; SecAgg is on intermediate per-round messages. Compression composes with both as a post-hoc lossy operator. The composition argument is laid out in `docs/privacy-analysis.md` §3.

## 2.3 Dataset and use case

**MIT-BIH Arrhythmia.** PhysioNet's MIT-BIH database [7] is the de facto benchmark for ECG arrhythmia classification. Sampling rate 360 Hz, two leads, beat-level annotations on ∼48 records. We use the first 12 records (100, 101, 103, 105, 106, 108, 109, 111, 112, 113, 114, 115), which `wfdb.dl_database` pulls without authentication.

**AAMI 5-class scheme.** Annotation symbols are mapped to the AAMI [8] 5-class grouping: $N$ (normal, including L/R/e/j), $S$ (supraventricular ectopic), $V$ (ventricular ectopic), $F$ (fusion), $Q$ (unknown / paced).

**Class imbalance.** Table 2.1 gives the realised distribution. The dataset is dominated by class $N$: it accounts for 97.1 % of the beats. Two classes (F, Q) have under 10 examples each; no clinical claim about them is possible from this data, but they are kept in the label space because they appear in the AAMI scheme.

Table 2.1: AAMI class distribution across the 12 MIT-BIH records, after beat-window segmentation (window size 360 samples). Class $N$ dominates with 97.1 % prevalence. Imbalance of this magnitude is the central technical fact of Chapter 7.

| Class | Count | Share |
|---|---|---|
| $N$ (normal) | 24,655 | 97.10 % |
| $V$ (ventricular ectopic) | 661 | 2.60 % |
| $S$ (supraventricular) | 62 | 0.24 % |
| $F$ (fusion) | 8 | 0.03 % |
| $Q$ (unknown / paced) | 7 | 0.03 % |
| Total | 25,393 | 100.00 % |

**Why ECG, why MIT-BIH.** ECG monitoring is the canonical wearable-IoT health workload; MIT-BIH is the canonical benchmark for it; the AAMI mapping is the canonical label scheme. A privacy-preserving FL framework that ignores the imbalance characteristic of this canonical workload is over-claiming.

# Chapter 3

# Threat model

The privacy stack of this framework is only as meaningful as the adversary it is built against. This chapter enumerates the assets being protected, the adversaries the framework defends against, the capabilities each is granted, which component of the privacy stack mitigates each threat, and the trust assumptions inherited from the deployment substrate. The threat model is the contract that the design choices in Chapter 4 are built to honour, and the limitations called out in Chapter 8 are the places where the contract is honoured only partially.

## 3.1 Assets being protected

Five categories of asset flow through the framework, each with its own sensitivity profile (Table 3.1). Raw waveforms and labels are both protected health information (PHI) under HIPAA and personal data under GDPR; per-client gradient updates leak training data via gradient-inversion attacks even though they are nominally derived data; the aggregated global model is the published artefact that membership-inference attacks operate on; patient and device identifiers are quasi-identifiers that must not be re-linkable to the local partitions.

Table 3.1: Assets the framework protects and their sensitivity. The privacy stack is responsible for preventing leakage of every row except patient identity, which is handled at the deployment layer (no identifier ever leaves the device in our protocol).

| Asset | Sensitivity |
|---|---|
| Raw ECG waveforms | PHI under HIPAA; highly sensitive |
| Local labels (arrhythmia class) | Diagnostic; highly sensitive |
| Per-client model updates ($\Delta w_i$) | Leaks training data via gradient inversion |
| Aggregated global model $w_t$ | Leaks via membership inference |
| Patient / device identity | Quasi-identifier; must not be re-linkable |

## 3.2 Adversaries and capabilities

Five adversary archetypes are considered. The first four are inside the contract; the fifth is explicitly out of scope and motivates a future-work item in Chapter 8.

### 3.2.1 A1 — Honest-but-curious aggregation server

**Capability.** Sees every message that reaches the server: per-client parameter updates, per-client metadata (number of training samples, training loss, latency telemetry), broadcast parameters, and round-level scheduling information.

**Behaviour.** Follows the protocol faithfully. Does not deviate from the aggregation rule, drop messages, or inject crafted messages.

**Goal.** Infer training-data properties of any individual client from its update — e.g. reconstruct individual training records from $\Delta w_i$ via gradient-inversion attacks, or learn whether a particular patient's beats appear in client $i$'s partition.

**Mitigation.** Pairwise-mask secure aggregation (Chapter 4, §4.7). Under SecAgg, the server's view of every per-round message is a uniformly random vector; only the sum across all clients de-randomises into the true aggregate of deltas. The server therefore has access to $\sum_i \Delta w_i$ but not to any individual $\Delta w_i$.

### 3.2.2 A2 — Curious-client coalition (size $t < N$)

**Capability.** A subset of clients colludes among themselves. They pool their private masks (or shared seeds in the prototype), their broadcast parameters, and the metadata they each receive from the server.

**Behaviour.** Trains honestly so as not to be detected by simple correctness checks (the framework's SecAgg is not currently correctness-verifiable from the server's side; that is future work).

**Goal.** Unmask a non-coalition client's update.

**Mitigation.** Pairwise masks between every unordered client pair $\{i, j\}$ mean that a coalition of size $t$ learns nothing about non-coalition updates as long as at least one mask between an honest pair survives the collusion. With $N = 4$ and pairwise masks the basic scheme withstands a coalition of up to $t = N - 2 = 2$ without compromise; with $t = N - 1 = 3$ the scheme leaks the remaining update because the only mask the honest client uses is one whose seed is held by a coalition member.

### 3.2.3 A3 — Network eavesdropper

**Capability.** Passive observation of the wire between client and server (e.g. an on-path attacker on Wi-Fi, an ISP, or a nation-state adversary at the network layer).

**Goal.** Same as A1, but with the additional restriction that the adversary does not control the server's storage or compute.

**Mitigation.** Transport-layer encryption (TLS, assumed; out of scope for the prototype but standard in any production deployment). The masked SecAgg payload is also a uniformly random vector to a network observer, providing defence-in-depth: even if TLS is subsequently broken, A3's view is no better than A1's, which SecAgg already addresses.

### 3.2.4 A4 — External adversary holding the released global model

**Capability.** Inspects, queries, or fine-tunes the released global model weights $w_T$. Does not see per-round messages. Has access to auxiliary information about all but one of the training records (the worst-case threat model for differential privacy).

**Goal.** Membership inference: "was patient $P$'s data in the training set?" or its quantitative variant, "what fraction of patients in this hospital appear in the training set?". Reconstruction: recover features of the training distribution beyond what is published.

**Mitigation.** **Differential privacy** with budget $(\varepsilon, \delta)$ at the per-record level. With $\varepsilon = 5$ and $\delta = 10^{-5}$ (the framework's default targets) the worst-case posterior shift on any single-record membership question is at most $e^{\varepsilon} \approx 148$, and empirical membership-inference accuracy is typically much closer to the 50% random-guess baseline. The cost of this guarantee on imbalanced healthcare data is the central finding of Chapter 7.

### 3.2.5 A5 — Malicious (Byzantine) client

**Capability.** Deviates arbitrarily from the protocol: submits crafted updates designed to poison the global model, install a backdoor, or break SecAgg by sending mismatched mask contributions.

**Goal.** Either degrade the global model's utility, install a trigger pattern that produces attacker-chosen outputs at inference time, or extract information from honest clients by exploiting protocol deviations.

**Out of scope.** The framework's threat model does not include A5. Robust aggregation rules (Krum, coordinate-wise median, trimmed mean, multi-Krum) and protocol-level integrity checks would be the natural defences; both are listed as future work in Chapter 8, §8.5. The simulation never produces an A5: the four clients are honest by construction.

## 3.3 Attack-surface coverage matrix

Table 3.2 maps the five canonical attacks against federated learning to which adversary mounts them and which layer of the privacy stack mitigates each one. The matrix is the most compact statement of the contract: every cell either references a mitigation or is explicitly out of scope. There are no implicit guarantees.

Table 3.2: Attack-surface coverage. Cells marked *SecAgg* are mitigated by the pairwise-mask layer, cells marked *DP* by differential privacy, cells marked *TLS+SecAgg* by both, and cells marked *OOS* are out of scope (Byzantine adversary A5).

| Attack | A1 | A2 | A3 | A4 | A5 |
|---|---|---|---|---|---|
| Gradient inversion | SecAgg | SecAgg | TLS+SecAgg | — | OOS |
| Per-update membership inference | SecAgg | SecAgg | TLS+SecAgg | — | OOS |
| Global-model membership inference | DP | DP | — | DP | OOS |
| Reconstruction from compressed update | SecAgg | SecAgg | TLS+SecAgg | — | OOS |
| Model poisoning / backdoor | — | — | — | — | OOS |

**Why compression does not weaken the matrix.** Top-$k$ + int8 compression sits between local DP-SGD and SecAgg masking on the client (Chapter 4, §4.7). Because SecAgg is applied *after* compression, the wire payload is still a uniformly random vector to A1/A2/A3, and the compressed update is never visible in the clear. DP's post-processing immunity ensures that compression does not weaken A4's bound on the released global model. Compression is a bandwidth optimisation, not a privacy defence; it is included in the matrix only to make explicit that it does not subtract from the matrix either.

## 3.4 Trust assumptions

The framework's guarantees are conditional on four trust assumptions inherited from the deployment substrate:

1. **TLS is in place between every client and the server** — standard mTLS in any production deployment. The simulation does not exercise this; in a real IoT deployment it is non-negotiable.

2. **The pseudo-random generator used to derive masks is cryptographically secure.** The prototype uses Python's `secrets` module for seed generation and a deterministic NumPy generator from that seed for mask material; the latter is sufficient for simulation but a production deployment would seed an authenticated stream cipher (e.g. AES-CTR) instead.

3. **No more than $N-2$ clients collude.** The basic pairwise-mask scheme has no Shamir secret-sharing fallback (a deliberate prototype simplification, called out in `src/privacy/secure_agg.py`), so a coalition of $N-1$ clients reconstructs the remaining client's update.

4. **Hardware-level side channels (timing, power, EM) on the IoT device are out of scope.** A real wearable deployment would also need hardware countermeasures; without them, an attacker with physical access to a device can extract the local model or training data regardless of what the FL protocol does.

## 3.5 What is explicitly out of scope

Beyond A5, four classes of guarantee are explicitly not provided:

- **Byzantine robustness / poisoning defences.** Future work (§8.5); the natural extension is robust aggregation (Krum, median, trimmed mean) composed with the existing privacy stack.

- **Hardware-level side channels.** Out of scope at the protocol layer; addressed at the deployment layer with hardware countermeasures.

- **Verifiable computation** (cryptographic proof that the server aggregated correctly). The framework trusts that the server runs the strategy code it claims to run; deployments in adversarial settings would add zk-SNARK or trusted-execution-environment-based attestation.

- **Free-rider detection.** A client that submits zero-update vectors round after round contributes nothing to the global model but receives the benefit of others' updates; the framework does not detect or penalise this. Reputation and contribution-tracking systems are the standard defence; we do not implement either.

These are flagged again in the conclusions chapter as natural extensions that fit cleanly on top of the existing privacy stack.

# Chapter 4

# Method

This chapter describes the implementation that produces every number in the thesis. All code lives under `src/` and `experiments/`; YAML configs in `configs/` are the canonical record of every run. The repository is at https://github.com/mapeke/privacy-preserving-fl-healthcare-iot.

## 4.1 Unified client interface (`src/client/flower_client.py`)

`FlowerHealthcareClient` subclasses `flwr.client.NumPyClient` with three methods. `fit` performs local training (optionally DP-SGD), computes the parameter delta against the broadcast parameters, optionally compresses it, optionally adds the SecAgg pairwise masks, and returns reconstructed parameters (broadcast + processed delta) so vanilla Flower strategies remain compatible. `evaluate` reports per-client test loss and accuracy. `get_parameters` serialises the model state into the wire format. The class takes a single `ClientConfig` dataclass with mutually-orthogonal fields `dp_config`, `secagg_session`, `compression_method`, each of which is `None` when the corresponding defense is off.

## 4.2 Data layer (`src/data/`)

`mitbih_loader.py` downloads MIT-BIH records via `wfdb.dl_database` into `data/cache/`, segments each record into 360-sample beat windows centered on R-peak annotations, applies per-record z-score normalisation, and maps annotation symbols to AAMI labels. A synthetic-ECG fallback exists for offline development; both real-data and synthetic-data runs go through the same downstream pipeline.

`partitioner.py` distributes beats across $N$ clients via Dirichlet sampling per class with concentration $\alpha$ (default 0.5; lower $\Rightarrow$ more skewed). For our 4-client setup the realised partition sizes are $(1907, 8621, 6947, 2840)$ — non-IID by construction.

`preprocessing.py` provides a Butterworth bandpass filter (0.5–40 Hz) for baseline-wander and powerline rejection, a stratified train/test split, and the `TorchEcgDataset` adapter.

## 4.3 Models (`src/models/`)

`EcgCNN` is a 3-block 1-D CNN with GroupNorm (used in place of BatchNorm for Opacus compatibility), 64 final channels, adaptive average pooling, and a 32-unit MLP head; ∼50k parameters. `LightweightEcgCNN` is a depthwise-separable variant at ∼5k parameters for resource-constrained edge inference. All experiments in this thesis use `EcgCNN`.

**ReLU non-inplace.** `nn.ReLU(inplace=True)` is incompatible with Opacus's per-sample gradient hooks (the in-place op writes through a backward-hook view). The exception is

swallowed by the Ray worker process in Flower's simulation backend, producing silently-flat training. Switching to non-inplace ReLU fixed this; see commit `40dd3f4`.

## 4.4 Differential privacy (`src/privacy/differential_privacy.py`)

`DPConfig` is the user-facing dataclass with `target_epsilon`, `target_delta`, `max_grad_norm` (the per-sample clip threshold $C$), optional `noise_multiplier`, and `accountant` ($\in$ {`rdp`, `gdp`, `prv`}). `attach_dp(model, optimizer, dataloader, config, epochs)` wraps the optimizer and dataloader via `opacus.PrivacyEngine`. When `noise_multiplier` is left unset Opacus auto-calibrates $\sigma$ to hit `target_epsilon` over the requested epochs; otherwise the user-supplied $\sigma$ is used. The returned handle exposes `get_epsilon(steps)` for per-round budget reporting.

## 4.5 Secure aggregation (`src/privacy/secure_agg.py`)

`setup_session(num_clients, seed_size_bytes=32, rng_seed=...)` generates one fresh secret seed per unordered client pair, stored in a `SecAggSession` object distributed by the experiment orchestrator. `derive_mask(seed, shape)` expands a pair seed into a Gaussian mask vector via NumPy's PCG64. `mask_update(update, client_id, session)` adds $+m_{ij}$ for $j > i$ and $-m_{ij}$ for $j < i$ to the client's update, so that summing all clients' masked updates cancels every mask exactly. `aggregate_masked_updates` performs the server-side sum on the strategy side (§4.7).

**Limitations explicitly accepted.** Pre-shared seeds rather than Diffie–Hellman; no Shamir secret-sharing (no dropout tolerance); float32 arithmetic rather than modular integer over a finite field. These are appropriate for a diploma simulation that wants to demonstrate the principle and measure end-to-end overhead, and they are flagged in the module docstring.

## 4.6 Compression (`src/privacy/compression.py`)

`compress(update, method, top_k_ratio, quant_bits)` dispatches over four modes: `none`, `top_k_only`, `quantize_only`, `top_k_quantize`. `top_k_sparsify` keeps the largest-magnitude $\lceil k|u| \rceil$ entries via $O(n)$ argpartition. `quantize_int8` performs min–max quantisation with a documented `(scale, zero_point)` pair; `decompress` reconstructs a dense update array (zeros where sparsified). The wire payload is a `CompressedUpdate` dataclass with `indices`, `values`, `original_shape`, and a `num_bytes()` method for bandwidth accounting.

## 4.7 Server strategy (`src/server/strategies.py`)

`PrivacyAwareFedAvg` subclasses Flower's `FedAvg`. With SecAgg disabled it falls through to standard weighted averaging. With SecAgg enabled, `configure_fit` caches the broadcast parameters; `aggregate_fit` recovers per-client deltas by subtracting them from each client's returned parameters, sums the deltas (where masks cancel by construction), divides by the number of participating clients, and applies the result on top of the cached broadcast. Robust scalar-metric aggregation (sample-weighted mean for numerics, first-occurrence for non-numerics) covers the case where Flower's default aggregator chokes on heterogeneous metric dictionaries.

## 4.8 Experiment infrastructure

Five entry points, each a thin CLI wrapper around `experiments/_common.py::build_experiment` + `run_federated_experiment`:

- `run_centralized.py` — pooled-data baseline.
- `run_federated.py` — vanilla FedAvg.
- `run_federated_dp.py` — FedAvg + DP-SGD.
- `run_federated_secagg.py` — FedAvg + SecAgg.
- `run_full_pipeline.py` — DP + SecAgg + compression.

Each writes `results/<name>/{run.log, resolved_config.yaml, history.json}` so every reported number can be traced back to its config.

## 4.9 Tests

42 pytest unit tests across seven modules; the most consequential is `test_secagg.py::test_masks_cancel_unde` which constructs four random updates, masks them, sums the masked vectors, and asserts that the result equals the unmasked sum to within $10^{-4}$ in $\ell_2$. This is contribution C2: the cryptographic correctness argument as a code-level invariant.

The test suite passes in 4.0s on the developer machine. `Opacus` is imported lazily inside `attach_dp` so the tests run without it installed; the test that asserts that lazy-import property is itself an invariant.

# Chapter 5

# Results I — Centralized and federated baselines

Setup: 12 MIT-BIH records (Table 2.1), 4 clients, Dirichlet $\alpha = 0.5$ partitions of sizes $(1907, 8621, 6947, 2840)$, 80/20 stratified train/test split. Configs: `configs/default.yaml`. Hardware: single NVIDIA RTX (mobile) GPU, CUDA 12.1, PyTorch 2.4.1. Seed 42.

## 5.1 Detection grid

Table 5.1 reports the headline accuracy for every configuration we run in this thesis. The remainder of this chapter establishes the top two rows; Chapters 6 and 7 establish the rest.

Table 5.1: Final test accuracy across all five experimental configurations. Federation is free; SecAgg is free; DP collapses to the majority-class prevalence; compression on top of DP does not move the needle further. Numbers are single-seed at seed 42 to match the thesis policy of "every number tied to one config row."

| Configuration | Test acc | Macro-F1 | Final $\varepsilon$ |
|---|---|---|---|
| Centralized (5 epochs) | **0.9878** | 0.36 | — |
| FedAvg, 4 clients, 10 rounds | **0.9880** | — | — |
| FedAvg + SecAgg, 10 rounds | 0.9860 | — | — |
| FedAvg + DP, 20 rounds, target $\varepsilon = 5$ | 0.9711 | — | 4.9955 |
| Full pipeline (DP+SecAgg+Comp), 30 rounds, target $\varepsilon = 8$ | 0.9711 | — | 7.9964 |
| Class-$N$ prevalence (= predict-all-N) | 0.9716 | 0.20 | — |

## 5.2 Centralized baseline

5 epochs of plain SGD on the pooled training set (20,315 beats), $\eta = 0.01$, momentum 0.9, weight decay $10^{-4}$, batch size 32. Final test accuracy 0.9878, macro-F1 0.3627. Per-class F1: $\{N{=}0.99,\ V{=}0.82,\ S{=}0.00,\ F{=}0.00,\ Q{=}0.00\}$. The model learns the two prevalent classes (N, V) and ignores the others. Macro-F1 is $\sim$0.36 because three of the five classes contribute zero, a structural property of the workload.

## 5.3 Vanilla FedAvg

10 rounds, 4 clients per round, 2 local epochs, batch 32, $\eta = 0.01$. Round-by-round test accuracy: $\{0.9711, 0.9711, 0.9758, 0.9840, 0.9840, 0.9724, 0.9831, 0.9864, 0.9862, 0.9880\}$.

**Finding 4.1 — Federation is free on this workload.** Final FedAvg accuracy 0.9880 vs. centralized 0.9878. The federated decomposition introduces zero accuracy cost up to seed variance. No special tuning, no proximal regularizer [10], no server-momentum — plain FedAvg suffices when client data is similar enough in distribution and the model is not over-parameterised.

This is the empirical baseline against which Chapters 6 and 7 measure the cost of the privacy stack. Any utility loss observed with SecAgg or DP enabled is attributable to the privacy mechanism, not to the federation itself.

## 5.4 Reproducibility check vs. published FL ECG results

Single-seed run; we did not run multi-seed for the headline figure due to GPU-time budget (each FedAvg run is ∼3 minutes on our hardware, but DP and full-pipeline runs are 30 minutes each; multi-seed across all five configurations is a future-work item, §8.5). Published FL ECG papers report top-1 accuracy in the 0.97–0.99 range on similar MIT-BIH subsets [9]; we are within that range. The thesis's contribution does not depend on matching any published absolute — it depends on the relative shape across the privacy stack rows.

# Chapter 6

# Results II — Secure aggregation

Configs: `configs/experiment_secagg.yaml`. SecAgg enabled, DP off, compression off. 10 rounds. Otherwise identical to Chapter 5.

## 6.1 Mask cancellation as a code-level invariant

**Finding 5.1 — Cryptographic correctness is mechanically verifiable.** The pairwise-mask correctness argument [4] is reduced to one assertion:

> *for* $N = 4$ *random updates of shape* $(10,)$, $\sum_{i=1}^{N} \text{mask_update}(u_i, i, \text{session}) = \sum_{i=1}^{N} u_i$ *to within* $10^{-4}$ $\ell_2$ *error.*

This lives in `tests/test_secagg.py::test_masks_cancel_under_aggregation` and runs in CI alongside the rest of the pytest suite. The same test verifies the dual property: each individual masked update is *not* close to the unmasked update (the masks are non-trivial; the cancellation only happens at the sum level).

The contribution here is not the test itself — it is one screenful of NumPy — but the discipline of writing the protocol's correctness argument as a checked invariant rather than as prose. A future change that breaks the protocol (e.g., an off-by-one in the $i < j$ vs. $i > j$ sign condition) fails the test loudly rather than silently producing biased aggregates.

## 6.2 End-to-end SecAgg run

Round-by-round test accuracy: $\{0.9711, 0.9711, 0.9711, 0.9776, 0.9837, 0.9840, 0.9844, 0.9848, 0.9846, 0.9860\}$. Final 0.9860.

**Finding 5.2 — SecAgg overhead is below run-to-run noise.** Vanilla FedAvg final 0.9880 vs. FedAvg + SecAgg final 0.9860 — a 0.20 pp gap. Per-class F1, train accuracy, and convergence trajectory all overlap with vanilla FedAvg's within a few hundredths. The 0.20 pp gap is consistent with float32 accumulation noise in the masking; it would shrink under integer modular arithmetic (future work, §8.5).

The combination of §6.1 and this finding is the case for SecAgg as a default in any FL deployment: the cryptographic correctness is a one-line CI check; the utility cost is below the noise floor; and the threat model that SecAgg addresses (honest-but-curious server with read access to the inbox) is realistic for any cloud-hosted aggregation server.

## 6.3 What SecAgg does not protect against

SecAgg blinds the server to per-client updates but does not blind it to the *aggregate*. An adversary holding the released global model $w_T$ can still mount membership-inference attacks

against training data; that threat is what differential privacy (Chapter 7) addresses. The two layers are complementary, and the thesis argues for running both, modulo the utility cost which Chapter 7 now measures.

# Chapter 7

# Results III — Differential privacy and the full pipeline

Configs: `configs/experiment_dp.yaml` and `configs/experiment_full.yaml`. DP enabled (Opacus PrivacyEngine, RDP accountant, `max_grad_norm = 1.0`). Otherwise identical to Chapter 5.

## 7.1 DP-SGD run

20 rounds, 1 local epoch per round, batch size 64, $\eta = 0.005$, target $\varepsilon = 5.0$, target $\delta = 10^{-5}$. Opacus auto-calibrates $\sigma$ to hit $\varepsilon = 4.9955$ (well within the target).

Round-by-round test accuracy: $\{0.9711, 0.9711, 0.9711, 0.9711, 0.9711, \ldots, 0.9711, 0.9711, 0.9711\}$. The trajectory is exactly flat to four decimal places across all 20 rounds.

Train metrics tell the inner story: train loss falls 0.5370 → 0.2233; train accuracy rises 0.9147 → 0.9712. The model is doing *something* during training; that something just happens to be "learn to predict N and only N."

## 7.2 Class-prior collapse

**Finding 6.1 — DP-SGD collapses to majority-class prediction.** Class $N$ prevalence in the test split: 4,934/5,078 = 0.9716. Final DP-SGD test accuracy: 0.9711. The two numbers agree to four decimals. The model is, with statistical certainty, the majority-class predictor.

This is not a learning-rate or noise-multiplier failure — the train accuracy converges to the same 0.9712, and the spent $\varepsilon$ is within the target. It is a workload-mismatch failure: DP-SGD's per-step Gaussian noise drowns out the gradient signal of the rare classes, which contribute ∼2.9 % of the data. Under sufficient noise, the empirical-risk-minimising policy on the training set is exactly: predict the majority. Opacus is doing what it advertises; the workload asks more of it than the budget will pay for.

**Why this matters editorially.** A casual reader of Table 5.1 sees "0.9711" beside "DP $\varepsilon = 5$" and concludes that DP costs the framework about 1.7 pp of accuracy — a number that would be publishable under the headline-accuracy convention dominant in the FL-privacy literature. The honest reading is that DP costs the model's ability to detect any rare class at all on this data. The 0.9711 number hides the failure precisely because the metric is overall accuracy on a heavily imbalanced label space.

The framework therefore reports macro-F1 alongside accuracy in `src/utils/metrics.py::compute_classi` the centralized run's macro-F1 of 0.36 (Chapter 5) drops to $\leq 0.20$ on the DP run, which would

have made the collapse visible if multi-seed F1 had been logged across rounds. We did not log it for the federated runs; we will (§8.5).

## 7.3 The full pipeline run

30 rounds, 1 local epoch, batch 64, $\eta = 0.005$, target $\varepsilon = 8.0$, target $\delta = 10^{-5}$, SecAgg enabled, top-$k$ + int8 compression with $k = 0.1$. Opacus auto-calibrates $\sigma$ for the looser $\varepsilon$, hits $\varepsilon = 7.9964$.

Round-by-round test accuracy: 0.9711 across all 30 rounds.

**Finding 6.2 — Compression on top of DP does not degrade further.** DP-only at $\varepsilon = 5$ converges to 0.9711. Full pipeline (DP + SecAgg + compression) at $\varepsilon = 8$ converges to 0.9711. The model is already at the floor that the workload's class prior imposes; there is nothing left to degrade by adding more layers. Loosening the privacy budget from $\varepsilon = 5$ to $\varepsilon = 8$ does not move the model off the floor either, which would otherwise be the natural follow-up question.

**Compositional implication.** For an operator constrained to deploy DP at $\varepsilon \leq 8$ on a class-imbalanced workload, adding SecAgg and compression is essentially free: they do not make the already-collapsed model any worse. This inverts the usual "every layer costs something" framing for the specific case where the bottom layer has already maxed out the utility cost.

## 7.4 Negotiating the cliff

The thesis does not propose a fix; the experimental contribution is to make the cliff visible. Three directions a follow-up paper would take:

1. **Class-weighted DP-SGD.** Apply per-class loss weights so that the gradient signal from the rare classes is amplified before clipping. Risks: changes the privacy accounting because the $\ell_2$ sensitivity of the loss is no longer 1.

2. **Per-class privacy budgets.** Decouple $\varepsilon$ for majority and minority classes, e.g. $\varepsilon_N = 1$, $\varepsilon_{S,V,F,Q} = 10$. Mathematically clean but practically awkward to deploy — the privacy guarantee is then conditional on the target class, which a deployment operator may find harder to reason about than a single $\varepsilon$.

3. **Different MIT-BIH records.** Records 200–234 contain more arrhythmias and reduce the imbalance from 97.1 % N to 82.1 % N. We re-ran all five configurations on this slice (§7.5 below) without retuning, both to test whether the cliff is dataset-conditional and to give the rare-class classifier a fairer chance. The deployment workload (a wearable monitoring an asymptomatic patient) is closer to the 100–115 imbalance than to 200–234, so the primary slice remains the more representative one.

## 7.5 Follow-up: records 200–234

To test whether the DP class-prior collapse (§7.2) is a property of DP-SGD itself or of the workload's extreme imbalance, we re-ran all five experimental configurations on records 200–234 — a 25-record slice with 61,818 beats and substantially more arrhythmia variety. No hyperparameters were retuned; configs are `configs/followup_records200/{default, experiment_dp, experiment_secagg, experiment_full}.yaml`. Table 7.1 contrasts the two slices' class distributions; Table 7.2 reports final accuracies side by side with the original Table 5.1.

Table 7.1: AAMI class distribution on records 100–115 (the primary slice, 25,393 beats) and records 200–234 (the follow-up, 61,818 beats). The follow-up reduces the majority-class share from 97.1 % to 82.1 % and gives all four rare classes between 1 % and 10 % of the data each.

| Class | 100–115 share | 200–234 share |
|---|---|---|
| $N$ (normal) | 97.10 % | 82.08 % |
| $S$ (supraventricular) | 0.24 % | 4.19 % |
| $V$ (ventricular ectopic) | 2.60 % | 9.53 % |
| $F$ (fusion) | 0.03 % | 1.28 % |
| $Q$ (unknown / paced) | 0.03 % | 2.93 % |

Table 7.2: Same five experiments, records 100–115 vs. records 200–234, same hyperparameters in both columns. The majority-class baseline (predict-all-N) is 97.16 % on 100–115 and 82.08 % on 200–234; the right-hand column is the gap from that baseline on the follow-up slice.

| Configuration | 100–115 acc | 200–234 acc | vs. majority |
|---|---|---|---|
| Centralized (5 epochs) | 0.9878 | **0.9389** | +11.81 pp |
| FedAvg (10 rounds) | 0.9880 | **0.9636** | +14.28 pp |
| FedAvg + SecAgg (10 rounds) | 0.9860 | **0.9558** | +13.50 pp |
| FedAvg + DP, $\varepsilon \approx 5$ (20 rounds) | 0.9711 | **0.8845** | +6.37 pp |
| Full pipeline, $\varepsilon \approx 8$ (30 rounds) | 0.9711 | **0.8176** | −0.32 pp |

**Finding 6.3 — The DP class-prior collapse is dataset-conditional.** On records 200–234, DP-SGD at $\varepsilon \approx 5$ converges to 0.8845 test accuracy, +6.37 pp above the 0.8208 majority-class baseline. The model has *not* collapsed: it is distinguishing rare-class beats. Compare to the original 100–115 result, where DP-SGD's 0.9711 matched class-$N$ prevalence to four decimals. Finding 6.1 (§7.2) is therefore not a property of DP-SGD but a property of *extreme* class imbalance composed with DP's per-step noise. The cliff exists; its location is workload-dependent.

**Finding 6.4 — The full pipeline collapses on moderate imbalance too.** The full-pipeline run (DP + SecAgg + top-$k$ + int8 at $\varepsilon \approx 8$) on records 200–234 converges to 0.8176, 0.32 pp *below* the 0.8208 majority-class baseline. Round-by-round accuracy oscillates between ∼65 % and ∼86 % over the 30 rounds; the run does not stabilise. Top-$k$ + int8 compression on top of DP-SGD's already-noisy gradients appears to push the regime back across the cliff that DP-SGD alone clears on this dataset. Loosening the budget from $\varepsilon = 5$ (DP-only) to $\varepsilon = 8$ (full pipeline) is not enough to compensate — the additional noise floor is the lossy compression, not the DP budget.

**What the follow-up changes editorially.** The Chapter 7 framing "DP collapses on imbalanced healthcare data" is true on records 100–115 but reads as a generalisation. The honest generalisation is: *DP-SGD's signal-to-noise ratio degrades sharply when the rare-class share approaches the per-step noise floor; the cliff is observable but its location depends on both the budget and the workload imbalance.* The full pipeline crosses the cliff on both slices, which is the more generalisable finding the framework supports.

**What the follow-up does not change.** We do not retune hyperparameters per dataset — the contribution is the relative shape across the privacy stack, not absolute accuracy on either slice. We do not claim records 200–234 is the "right" slice; the deployment workload (a wearable monitoring an asymptomatic patient) is closer to the 100–115 imbalance. Both slices are useful:

100–115 surfaces the cliff at the dataset extreme, 200–234 shows where on the class-balance axis it sits.

# Chapter 8

# Discussion

## 8.1 What the three findings add up to

- **Federation (§5.3).** Free on this workload.
- **Secure aggregation (§6.1, §6.2).** Free up to fp32 mask-cancellation noise; cryptographic correctness is a one-line CI invariant.
- **Differential privacy (§7.2).** The privacy-utility cliff is real and is the central pitfall of DP-SGD on imbalanced healthcare data. The DP run's headline accuracy looks reasonable (0.9711) precisely because the metric matches the cliff: predicting the majority class scores 0.9716.
- **Dataset dependence (§7.5).** Finding 3 is conditional on *extreme* class imbalance: on records 200–234 (82 % N rather than 97 % N) DP-SGD alone retains a +6.37 pp rare-class signal, while the full pipeline still collapses. The cliff is a property of the workload composed with the privacy stack, not of DP-SGD per se; the framework is what makes that property visible.

The thesis-level recommendation: any privacy-preserving FL paper for imbalanced healthcare data should report majority-class baseline alongside overall accuracy and macro-F1 alongside top-1; the privacy/utility tradeoff graph is misleading without them. Our framework contributes both the mechanism that produces the data and the protocol that surfaces the cliff.

## 8.2 Likely reviewer objections

**"You only ran each configuration at one seed."** True. Vanilla FedAvg, SecAgg, and DP are seed-stable on this dataset to within roughly 0.5 pp accuracy according to spot-checks during development. The DP collapse is structural rather than seed-dependent — it is the model converging to the majority-class predictor, which the same training process produces from any random initialisation. Multi-seed across all configurations is the first item of §8.5.

**"You only used 12 records."** The primary experiments use records 100–115 (the `wfdb.dl_database` default). §7.5 re-runs all five experiments on records 200–234 (25 records, 61,818 beats, 82.1 % N) without retuning. The qualitative finding holds with one refinement: DP-SGD alone clears the cliff on records 200–234 but the full pipeline still collapses. The 12-record default is enough to *surface* the cliff; the 25-record follow-up is what places it on the class-balance axis.

**"You did not compare to FedProx or SCAFFOLD."** The contribution is layering existing privacy primitives on top of FedAvg, not optimising the FL aggregation itself. FedAvg achieves 0.988 on this workload — approximately the centralized baseline. There is no headroom for a stronger FL aggregator to demonstrate on this data. A different workload (more clients, more heterogeneity) would justify the comparison.

**"Your SecAgg is not Bonawitz et al."** Acknowledged. The prototype uses pre-shared seeds rather than Diffie–Hellman, no Shamir secret-sharing for dropout tolerance, and float32 arithmetic. These limitations are listed in the module docstring, in `docs/privacy-analysis.md` §2, and at the start of §6. The cryptographic correctness invariant is the only contribution we claim from the SecAgg layer; that survives the simplifications.

**"Why not MIMIC-IV / PhysioNet challenge data / X?"** MIT-BIH is the canonical ECG benchmark; all other datasets we considered (MIMIC-IV vital signs, the PhysioNet 2020 12-lead challenge) introduce either feature heterogeneity or scale that the framework does not currently model. The framework is dataset-agnostic at the loader level (`src/data/`); switching to MIMIC-IV is a one-file extension and is listed in §8.5.

## 8.3 Practical implications for deployment

- **Run SecAgg by default.** Free utility cost (§6.2), meaningful threat coverage (honest-but-curious server, per-update tracing), and the correctness check is one unit test.
- **DP-SGD on imbalanced clinical data is non-trivial.** The headline-accuracy number can mask total class-prior collapse. Always report majority-class baseline and macro-F1.
- **Compression is free in the post-DP regime.** If the model is already at the privacy floor, top-$k$ + int8 compression buys bandwidth at no further utility cost. If the model is *not* at the floor, compression introduces additional approximation error and must be measured explicitly.

## 8.4 What we deliberately did not test

- Byzantine / poisoning attacks. The threat model is honest-but-curious server plus model-recipient adversary.
- Real network-level performance. The Flower simulation is in-process.
- Patient-level privacy guarantees (vs. record-level). Each MIT-BIH record corresponds to one patient in our partition, but the DP $\varepsilon$ is per-record, not per-patient. Patient-level $\varepsilon$ via group privacy is an extension we list but did not implement.

## 8.5 Future work

1. **Multi-seed runs across all five configurations** to put error bars on Table 5.1. Roughly 4 hours of GPU time per 3-seed sweep; limiting factor is the full-pipeline run (30 min/seed).
2. **Class-weighted DP-SGD** to test the proposed mitigation in §7.4 on the records 200–234 slice, where DP-SGD alone clears the cliff but the full pipeline does not. Records 200–234 already lifted the rare-class signal; class-weighted loss is the next dial.
3. **Modular-integer SecAgg** to remove the float32 cancellation noise observed in §6.2 and bring the SecAgg overhead to exactly zero.

4. **Byzantine-robust aggregation** (Krum, median, trimmed mean) composed with the existing privacy stack.
5. **MIMIC-IV vital-signs port** to test whether the class-prior collapse generalises to a different healthcare modality with different label distribution.
6. **Diffie–Hellman key exchange + Shamir secret-sharing** to upgrade the SecAgg prototype to a dropout-tolerant deployment-grade protocol.

# Chapter 9

# Conclusion

We built a layered-defense privacy-preserving federated learning framework for ECG arrhythmia classification on MIT-BIH, with three independently switchable defenses (DP-SGD, pairwise-mask SecAgg, top-$k$ + int8 compression) on top of FedAvg. Five configurations were benchmarked on real PhysioNet data with four simulated wearable IoT clients.

Three findings define the thesis:

1. **Federation is free** on this workload — vanilla FedAvg matches centralized accuracy within 0.02 pp. The federated decomposition introduces no measurable utility cost.

2. **Secure aggregation is free** up to float32 mask-cancellation noise — 0.20 pp gap vs. vanilla FedAvg, well below run-to-run variance. The cryptographic correctness property is encoded as a unit test that runs in CI; the protocol is mechanically verified, not just argued.

3. **DP-SGD collapses to majority-class prediction** on this imbalanced workload. The final accuracy at $\varepsilon = 5$ is 0.9711, which equals the prevalence of class $N$ (Normal) to four decimals — the model is, with statistical certainty, the majority-class predictor. Adding compression on top of DP does not degrade further because there is nothing left to degrade. A follow-up on records 200–234 (§7.5) shows that the collapse is *dataset-conditional*: on moderate imbalance (82 % N) DP-SGD alone retains a +6.37 pp signal over the majority baseline, but the full pipeline still collapses at $\varepsilon = 8$. The sharpness of the cliff is a property of the workload imbalance; its existence is a property of the privacy stack composition.

The single recommendation this thesis makes to the privacy-preserving-FL-for-healthcare community: report majority-class baseline and macro-F1 alongside overall accuracy. The privacy/utility tradeoff curve in any imbalanced-data regime is meaningless without them; in our experiments it is the only thing standing between an honest "DP collapsed the model" and a reportable "DP costs 1.7 pp."

The protocol and infrastructure to do so are contributed in `src/` and `experiments/`, all driven by committed YAML configurations. Every number in this document can be reproduced with `python -m experiments.run_*` on the corresponding config.

Future work (§8.5) is multi-seed validation, records 200–234, modular-integer SecAgg, and a MIMIC-IV port.

# Bibliography


[1] H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. *Communication-Efficient Learning of Deep Networks from Decentralized Data.* AISTATS 2017, pp. 1273–1282.

[2] Martín Abadi, Andy Chu, Ian Goodfellow, H. Brendan McMahan, Ilya Mironov, Kunal Talwar, and Li Zhang. *Deep Learning with Differential Privacy.* ACM CCS 2016, pp. 308–318.

[3] Ilya Mironov. *Rényi Differential Privacy.* IEEE Computer Security Foundations Symposium (CSF) 2017, pp. 263–275.

[4] Keith Bonawitz, Vladimir Ivanov, Ben Kreuter, Antonio Marcedone, H. Brendan McMahan, Sarvar Patel, Daniel Ramage, Aaron Segal, and Karn Seth. *Practical Secure Aggregation for Privacy-Preserving Machine Learning.* ACM CCS 2017, pp. 1175–1191.

[5] Ashkan Yousefpour, Igor Shilov, Alexandre Sablayrolles, Davide Testuggine, Karthik Prasad, Mani Malek, John Nguyen, Sayan Ghosh, Akash Bharadwaj, Jessica Zhao, Graham Cormode, and Ilya Mironov. *Opacus: User-Friendly Differential Privacy Library in PyTorch.* NeurIPS Privacy in ML Workshop, 2021.

[6] Daniel J. Beutel, Taner Topal, Akhil Mathur, Xinchi Qiu, Javier Fernandez-Marques, Yan Gao, Lorenzo Sani, Kwing Hei Li, Titouan Parcollet, Pedro Porto Buarque de Gusmão, and Nicholas D. Lane. *Flower: A Friendly Federated Learning Research Framework.* arXiv:2007.14390, 2020.

[7] George B. Moody and Roger G. Mark. *The Impact of the MIT-BIH Arrhythmia Database.* IEEE Engineering in Medicine and Biology Magazine, 20(3):45–50, 2001.

[8] Association for the Advancement of Medical Instrumentation. *Testing and Reporting Performance Results of Cardiac Rhythm and ST Segment Measurement Algorithms (ANSI/AAMI EC57)*, 1998.

[9] Mohammad Kachuee, Shayan Fazeli, and Majid Sarrafzadeh. *ECG Heartbeat Classification: A Deep Transferable Representation.* IEEE International Conference on Healthcare Informatics (ICHI) 2018, pp. 443–444.

[10] Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith. *Federated Optimization in Heterogeneous Networks.* MLSys 2020.

[11] Jakub Konečný, H. Brendan McMahan, Felix X. Yu, Peter Richtárik, Ananda Theertha Suresh, and Dave Bacon. *Federated Learning: Strategies for Improving Communication Efficiency.* NeurIPS Workshop on Private Multi-Party Machine Learning, 2016.

[12] Siwon Kim, Kukjin Choi, Hyun-Soo Choi, Byunghan Lee, and Sungroh Yoon. *Towards a Rigorous Evaluation of Time-series Anomaly Detection.* AAAI 2022, pp. 7194–7201.

[13] Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. *Membership Inference Attacks Against Machine Learning Models*. IEEE Symposium on Security and Privacy (S&P) 2017, pp. 3–18.